Material Property Symbolic-Regression Candidate Evaluation

An executable paceval C++ demonstration of dimensioned, reusable materials-property expressions



Cover illustration: materials, symbolic candidate equations, reusable execution and synthetic model evaluation.

Research outcome. Using the supplied paceval 4.25 public C/C++ facade and a compatible Linux engine, the project constructed 1,000 reusable expression graphs. A deterministic workload executed 60,000 paceval scalar evaluations and 32,000 paceval batch evaluations, all agreeing with separately programmed native C++ formula outputs within 2×10^{-9} GPa.

Independent audit. The 60 exported selected-candidate predictions were also independently recomputed using 80-significant-digit decimal arithmetic, with all 60 within 2×10^{-10} GPa. The independently coded decimal oracle checks output arithmetic; it does not imply paceval itself computes to 80-digit precision.

Scope. The data are synthetic, the equation families are predeclared and their coefficients are not fitted. This is a mathematical candidate-evaluation demonstration, not scientific discovery of a new law, measured-property validation, or demonstrated superiority to existing material-science models.

1. Research question and execution architecture

The originating WP-019 research proposal asks whether mathematical formulas used during material-property symbolic-regression candidate evaluation can be represented as reusable execution graphs and independently checked. This implementation tests that numerical interface with a controlled, fully specified synthetic calculation kernel.

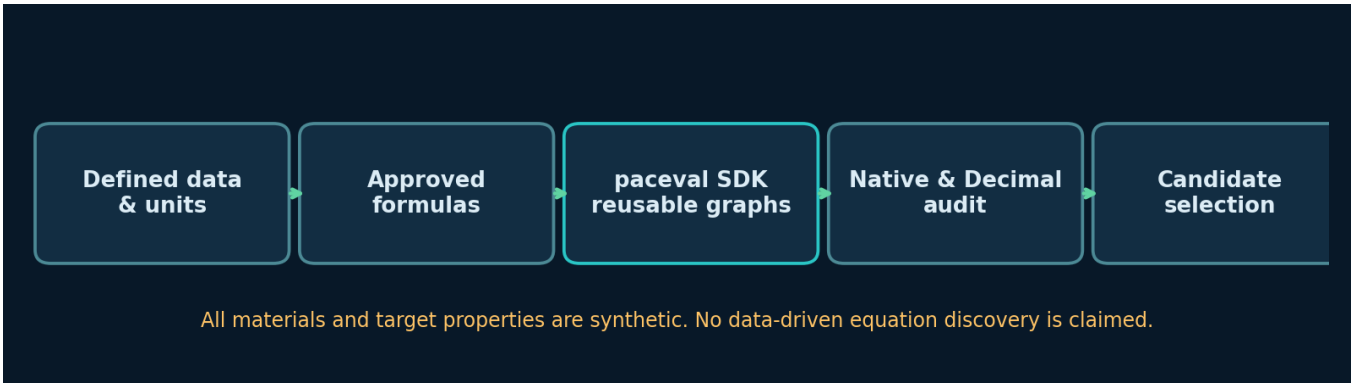


Figure 1. Dimensioned input contracts, fixed candidate grammar, persistent paceval computation graphs and independent checks.

The host defines the material-like descriptor x, absolute temperature T, candidate-parameter units, fixed sample splits and candidate registry. paceval receives each approved expression and evaluates new numeric input pairs through an existing graph handle. A separate native C++ evaluator checks every scalar and batch result.

Boundary of responsibility. paceval evaluates formulas. It does not select physical descriptors, enforce general dimensional algebra, discover equations, estimate coefficients, supply experimental labels or certify a thermodynamic model. Those functions require independent scientific software and domain review.

2. Mathematical contracts and units

The original proposal illustrates E = a + b x + c exp(-d/T). Here E is a synthetic modulus-like scalar measured in GPa, x is a dimensionless material-like descriptor and T is absolute temperature (kelvin). The parameters a, b, c and optional e are in GPa, while d is in kelvin; the literal 300 in T/300 means a fixed reference temperature of 300 K. Consequently every exponential and rational argument is dimensionless and each additive term has GPa units.

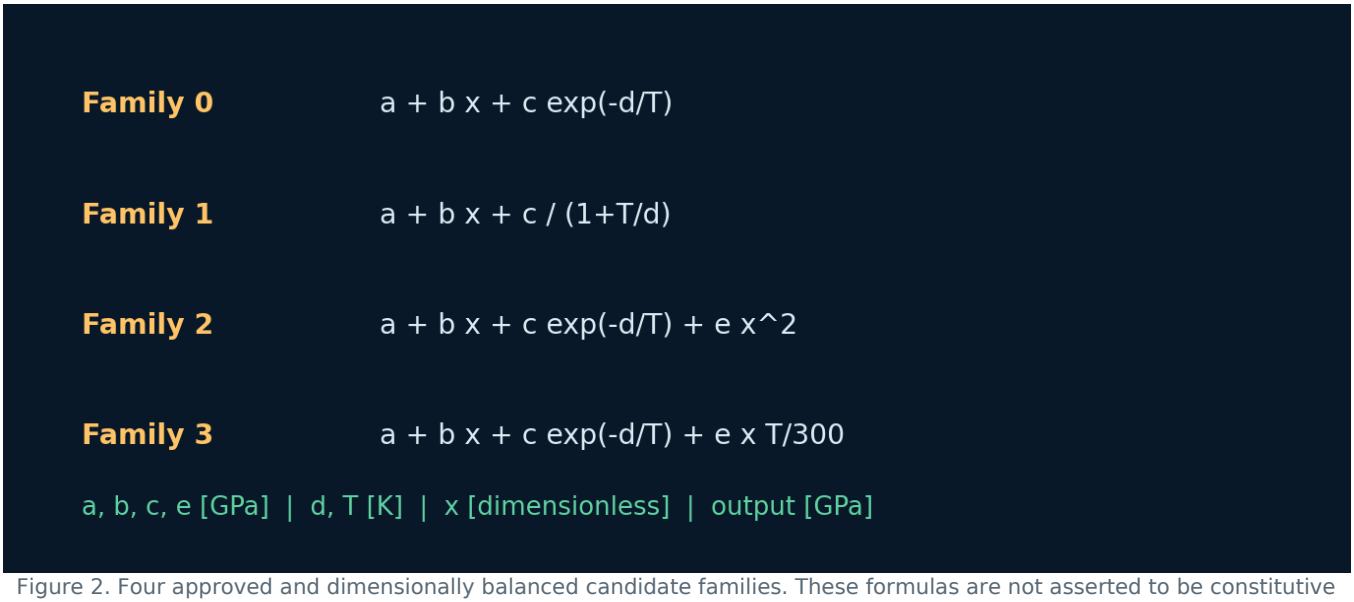


Figure 2. Four approved and dimensionally balanced candidate families. These formulas are not asserted to be constitutive laws.

Allowed input domain: x in [0,1], T in the deterministic range 260-890 K and d greater than zero. The constructed source family is family 0 with a = 100 GPa, b = 25 GPa, c = 18 GPa and d = 500 K. A bounded sinusoidal discrepancy is added to form synthetic observations. The generator inserts this known family as candidate 0, thus successful selection is not novel equation discovery.

The dimensional contract is enforced by the fixed, approved family registry and units of its coefficients. It is not a general-purpose symbolic dimensional-analysis engine; untrusted arbitrary mathematical strings are not admitted.

3. Synthetic samples and fixed partitions

The program creates 60 synthetic samples with a deterministic integer recurrence for x and T and a deterministic small sinusoidal additive discrepancy in GPa. The chronological order of IDs defines 36 training, 12 validation and 12 held-out test samples. This split is a fixed software-testing partition, not an independent real-material test or a benchmark-approved Matbench fold.

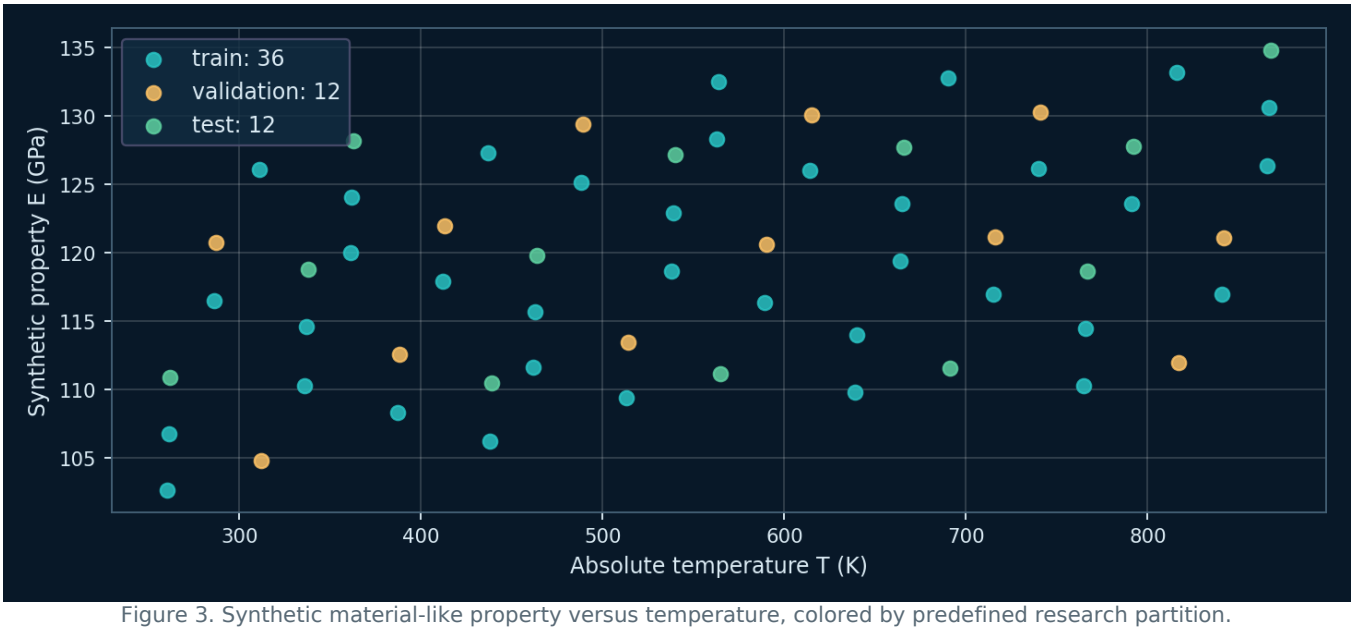


Figure 3. Synthetic material-like property versus temperature, colored by predefined research partition.

Partition	Purpose
Training (36 samples)	Calculate and report training RMSE for every candidate. No coefficients are optimized.
Validation (12 samples)	Select the candidate with the lowest validation RMSE; candidate ID resolves exact ties.
Held-out test (12 samples)	Report the selected candidate test RMSE only; no reselection using test results.

No published materials dataset is included and no materials measurements are claimed. For later external validation, Matbench and NIST JARVIS-DFT offer task-specific data, but property definitions, licensing, descriptor design and original official splits must be respected.

4. Evaluation of 1,000 versioned candidates

Exactly 1,000 predeclared and parameterized candidate formulas are constructed using a deterministic generator. Every candidate is created as a separate paceval computation object (handle), then evaluated on all 60 feature pairs without rebuilding its expression. For each graph, 32 additional batched values are checked using the SDK extended-result API. Each expression is also evaluated by an independently coded native function with matched mathematical semantics.

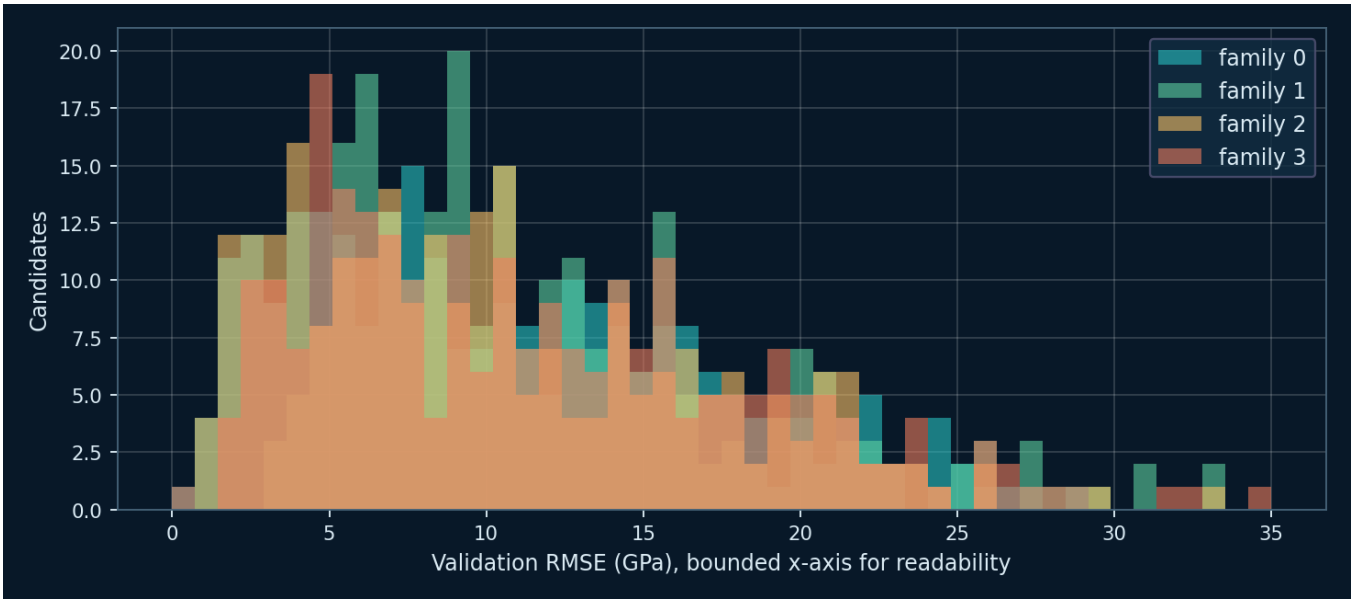


Figure 4. Distribution of validation errors over 1,000 candidates in the synthetic experiment. The displayed x range is limited for readability.

Candidate selection uses validation RMSE, not in-sample fit or test error. Candidate 0, the intentionally supplied data-generating family, was selected. No symbolic search or coefficient-fitting algorithm is part of this release. The ranking file exposes training/validation scores and a family identifier, not an opportunistically optimized test leaderboard.

A general symbolic-regression framework would require candidate enumeration or mutation, parameter estimation, constraints, search complexity penalties, cross-validation and independent checks for physical validity. This package implements the reusable candidate-evaluation stage on which such a workflow could rely.

5. Selected candidate and arithmetic agreement

The selected candidate is formula family 0 (candidate ID 0) with the same coefficients used in generating the underlying noiseless synthetic signal. It achieved a validation RMSE of approximately 0.08605 GPa and a held-out synthetic test RMSE of approximately 0.08941 GPa. These numerical quantities describe only the engineered synthetic workload and cannot be extrapolated to materials in nature.

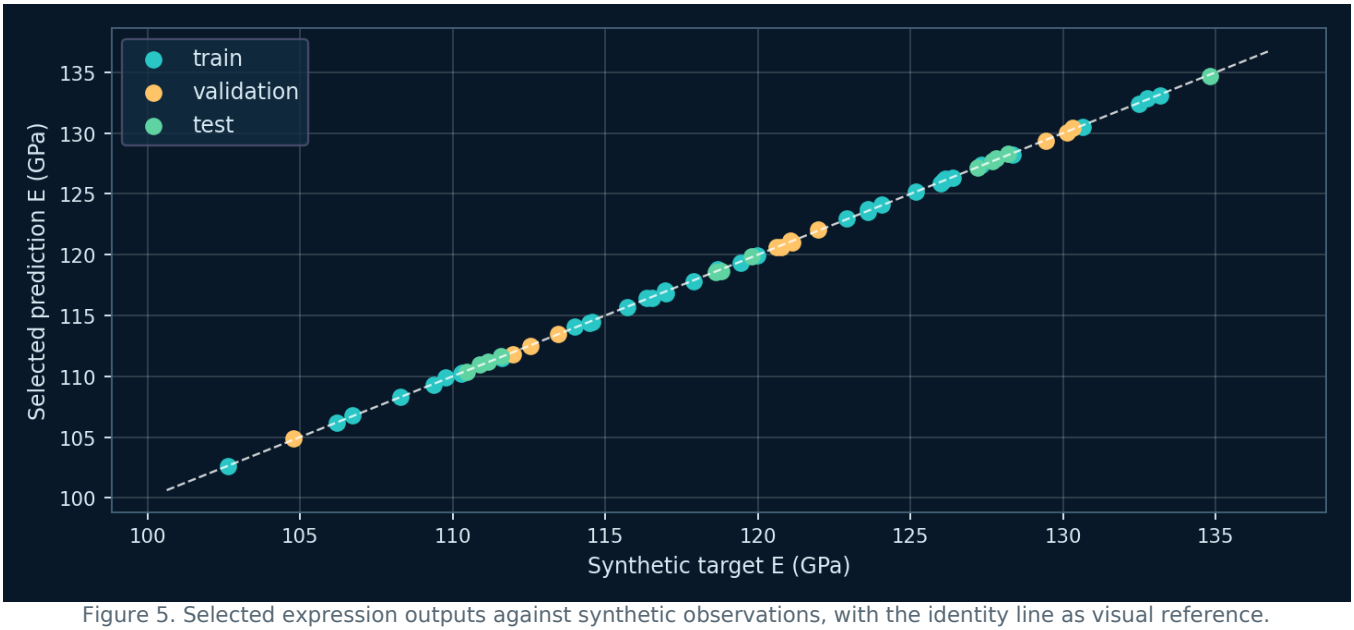


Figure 5. Selected expression outputs against synthetic observations, with the identity line as visual reference.

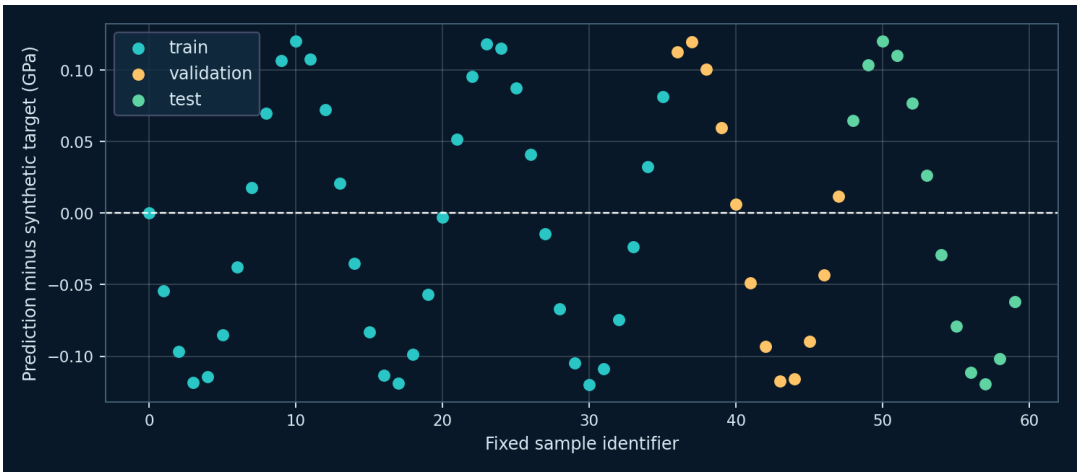


Figure 6. Signed residuals for the selected candidate, displayed against fixed sample identifiers.

6. SDK execution and independent numerical audit

Check	Reproduced observation
CMake and CTest	Build succeeded; 1/1 SDK integration tests passed
Reusable expression graphs	1,000 created and reused
paceval scalar versus native C++	60,000/60,000 within 2e-9 GPa
paceval batch versus native C++	32,000/32,000 within 2e-9 GPa
Selected native output versus Decimal	60/60 within 2e-10 GPa
Decimal working precision	80 significant decimal digits
Maximum selected Decimal discrepancy	2.078e-14 GPa

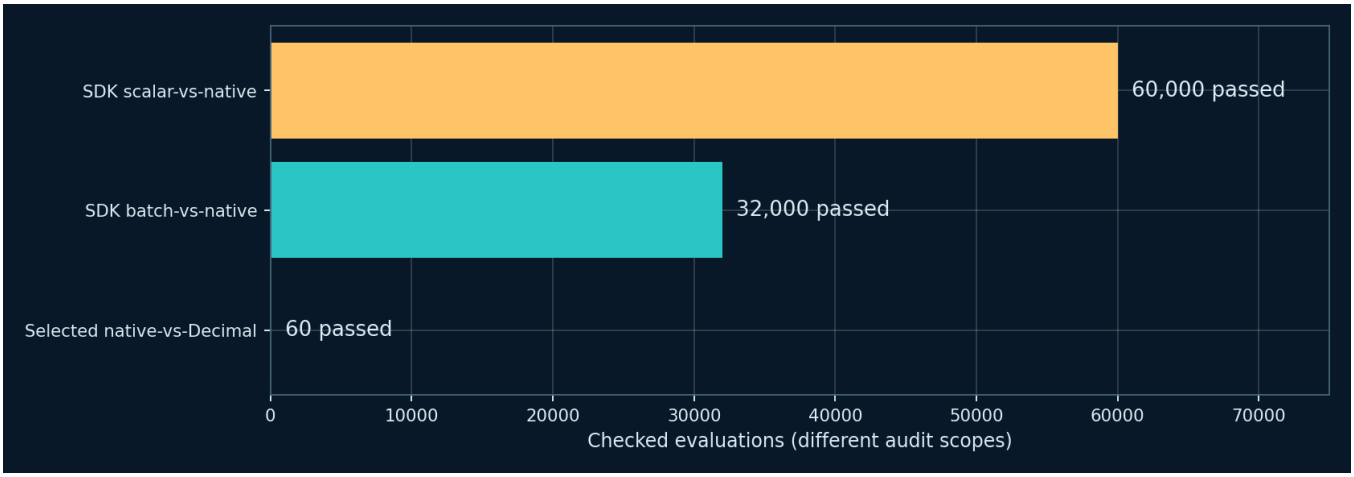


Figure 7. Different verification scopes. Decimal checks concern the selected candidate only.

The C++ program also checks the return convention of the paceval batch API, all returned per-item error flags, finite numerical outputs and handle lifecycle. The Python Decimal oracle separately reconstructs the chosen expression from printed binary64-derived decimal strings. These finite tests do not prove arithmetic correctness for all possible inputs.

7. Sensitivity, reproducibility and extensions

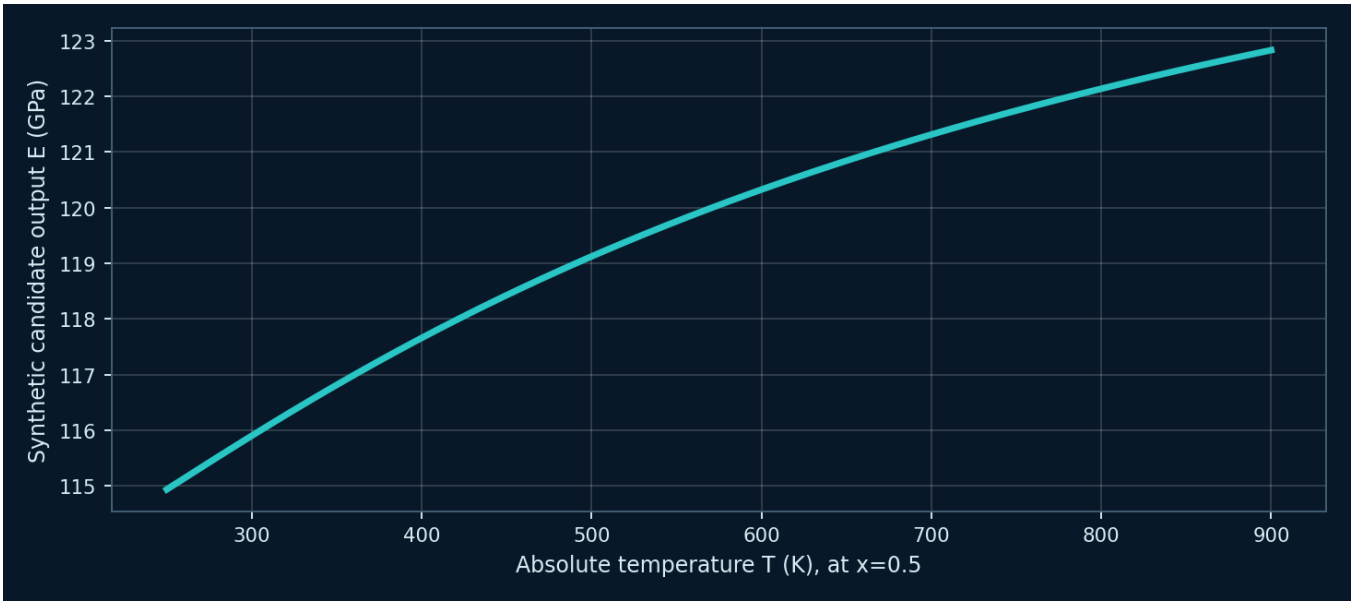


Figure 8. Temperature sweep of the selected synthetic formula with dimensionless descriptor fixed at x=0.5.

The figure shows direct reuse of the approved expression over a temperature sweep. It does not imply thermodynamic consistency, phase behavior, stability, monotonicity in all physical regimes or an observed temperature dependence in any real material.

Rebuild. Use **cmake -S src -B build -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so**, followed by **cmake --build build** and **ctest --test-dir build --output-on-failure**. Run **./build/material_candidates --candidates 1000 --out-prefix reproduced/wp019** after creating that output folder. Run **python3 src/high_precision_reference.py reproduced/wp019_selected.csv reproduced/audit.json**. Matplotlib and NumPy are needed to regenerate figures, ReportLab/Pillow to regenerate this PDF.

For a future real-data study, use an exact Matbench task such as log10(K/GPa) with its official folds, document the feature-engineering pipeline, and avoid applying the synthetic modulus equations directly to incompatible logarithmic targets. Alternatively, use JARVIS-DFT with carefully specified data extraction, licensing and splits. Compare candidate ranking, dimensional consistency, out-of-domain behavior and residuals independently.

8. Interpretation, limitations and references

Supported finding. A C++ program using the original paceval C/C++ facade and a compatible Linux shared engine executed 1,000 reusable material-property candidate graphs. It verified 60,000 scalar and 32,000 batch values against native C++ formulas and independently rechecked 60 selected outputs with 80-digit decimal arithmetic. This is a successful numerical feasibility result for predefined candidate evaluation.

Not established. This program does not discover new materials laws, identify material chemical compositions, fit coefficients, enforce a complete physics grammar, use NIST or Matbench measurements, test extrapolation on real materials, or establish physical predictiveness. No performance/processor comparisons or hardware-related advantage are reported.

Software provenance. The supplied original paceval SDK 4.25 public facade files are included as src/paceval_main.cpp and src/paceval_main.h, with only the CPP local include path normalized. Their original copyright and AGPL notices are retained. The supplied third-party/binary shared library is used as an external dependency during research testing and is not bundled in the GitHub ZIP. Verify applicable licensing before publication or distribution.

1. paceval mathematical engine, C/C++ API
https://paceval.com/api/

2. paceval public source repository
https://github.com/paceval/paceval

3. Matbench official project documentation
https://docs.materialsproject.org/services/ml-and-ai-applications/matbench

4. Dunn et al. (2020), Matbench benchmark, npj Computational Materials
https://www.nature.com/articles/s41524-020-00406-3

5. NIST JARVIS-DFT material property repository
https://www.nist.gov/programs-projects/jarvis-dft

6. JARVIS-DB public download overview
https://jarvis-materials-design.github.io/dbdocs/thedownloads/

7. Physics-guided symbolic regression in constitutive modeling (2025)
https://doi.org/10.1139/cgj-2025-0201

Conclusion. The candidate-evaluation kernel proposed in WP-019 has been implemented, directly executed with paceval, numerically audited and packaged for independent reproduction. The result supports integration of reusable mathematical expression evaluation into a future materials symbolic-regression workflow, while leaving equation search and scientific materials validation explicitly open.